

EXPERIMENTAL — Context Compaction

Bound self-managed in-memory history without changing service-managed conversations

Source: <https://learn.microsoft.com/en-us/agent-framework/concepts/agents/conversations/compaction?tabs=python>

Why compact at all?

- **Context window** — Long conversations eventually exceed model limits
- **Cost** — Larger prompts consume more input tokens
- **Latency** — More input tokens can slow each response
- **Trade-off** — Less history can also remove facts the agent needed

How it works

- Compaction operates on a flat list of Message objects
- Messages are annotated with lightweight group metadata
- Strategies mutate those annotations in place to mark groups as excluded before the message list is projected to the model

Choosing a strategy

Strategy	Aggressiveness	Preserves context	Requires LLM	Best for
ToolResultCompactionStrategy	Low	High — collapses tool results into summary messages	No	Reclaiming space from verbose tool output
SelectiveToolCallCompactionStrategy	Low–Medium	Medium — fully excludes old tool-call groups	No	Removing tool history when results are no longer needed
SummarizationStrategy	Medium	Medium — replaces history with a summary	Yes	Long conversations where context matters
SlidingWindowStrategy	High	Low — drops oldest groups	No	Hard group-count limits
TruncationStrategy	High	Low — drops oldest groups	No	Emergency message- or token-budget backstops
TokenBudgetComposedStrategy	Configurable	Depends on child strategies	Depends	Layered compaction with a token-budget goal and multiple fallbacks

Applicability

Applies

- Self-managed message history
- In-memory history agents
- Your process sends the full message list

No effect

- Foundry Agents
- Responses API with store=true
- Copilot Studio conversations

EXPERIMENTAL Strategy ladder (least destructive to most)

Day 3 · Module 3 · EXPERIMENTAL

1. **Collapse tool results** — ToolResultCompactionStrategy

2. **Exclude selected calls** — SelectiveToolCallCompactionStrategy

3. **Summarize history** — SummarizationStrategy uses a model

4. **Keep a recent window** — SlidingWindowStrategy

5. **Hard backstop** — TruncationStrategy

6. **Compose a budget** — TokenBudgetComposedStrategy

Grounded Python: keep recent groups

```
from agent_framework import SlidingWindowStrategy

# EXPERIMENTAL: keep the newest 20 non-system groups.
strategy = SlidingWindowStrategy(
    keep_last_groups=20,
)
```

D E M O

~4 min

SlidingWindowStrategy pruning old messages, live

Build a long local message list, print the group count, apply the previous slide's exact `SlidingWindowStrategy(keep_last_groups=20)`, and print the group count again — the EXPERIMENTAL ladder's gentlest rung, proven on real data with no model call required.

Runbook: [demos/day3/module-3-demo-1-sliding-window.md](#)

Compose by token budget

Collapse

Reclaim verbose tool-result space

Summarize

Preserve decisions with an extra model call

Window

Keep the most recent groups

Fallback

Exclude oldest groups until the token target is met

Measure what compaction changes

Measure	Before / after question
Input tokens	Did the strategy reach the intended budget?
Latency and cost	Did the reduction materially help?
Task quality	Did answers or tool choices regress?
Fact retention	Are user decisions and constraints preserved?
Tool integrity	Are tool call/result groups still atomic?

EXPERIMENTAL takeaways



You use compaction only for self-managed in-memory history.



You do not expect it to affect Foundry Agents, stored Responses, or Copilot Studio.



You start gentle, preserve atomic groups, and keep a hard backstop.



You evaluate quality as well as tokens, cost, and latency.



You do not make this experimental feature a core lab requirement.